

AYE
TECH HUB

• INDUSTRIAL ROBOTICS • LESSON 04

ABB Robot Systems and Programming

IRB Series · RobotStudio · RAPID · MoveL · MoveJ · I/O

Lesson 04 of 30 | Industrial Robotics Program

Awet G. Nway

Founder, AYE Tech Hub | Industrial Robotics & Automation Specialist

- ABB IRB robot families — which model for which application
- The IRC5 controller: architecture, FlexPendant and inputs/outputs
- RobotStudio — ABB's offline programming and simulation software
- Writing your first RAPID program: MoveJ, MoveL, MoveC and I/O
- Speed, zone data, work objects and tool data in RAPID

ABB Robotics — The World's Largest Robot Manufacturer

ABB (Asea Brown Boveri) is a Swiss-Swedish multinational and the world's largest supplier of industrial robots, with over 400,000 robots installed globally. ABB robots are used in automotive, electronics, pharmaceuticals, food and beverage, and metal fabrication industries worldwide. Learning ABB's RAPID programming language is the most transferable industrial robotics skill you can acquire — it is used in every country where manufacturing is present.

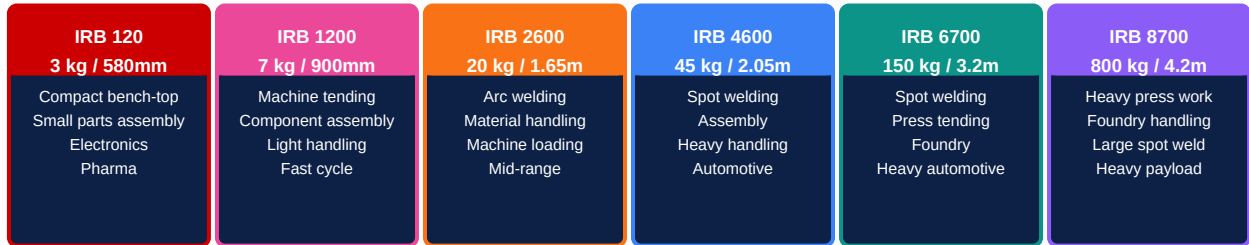


Figure 1 — ABB IRB Robot Families by Payload and Reach

Robot Model	Payload	Reach	Axes	Key Application	Competing Series
IRB 120	3 kg	580 mm	6	Lab automation, pharma, small electronics	KUKA KR 3, Fanuc LR Mate
IRB 1200	7 kg	900 mm	6	Machine tending, assembly, material handling	KUKA KR 6, Fanuc M-10iA
IRB 2600	20 kg	1 650 mm	6	Arc welding, machine tending, mid-payload	KUKA KR 16, Fanuc R-2000
IRB 4600	45 kg	2 050 mm	6	Spot welding, automotive assembly, handling	KUKA KR 60, Yaskawa MH50
IRB 6700	150 kg	3 200 mm	6	Press tending, foundry, spot welding, heavy	KUKA KR 120, Fanuc R-2000
IRB 8700	800 kg	4 200 mm	6	Heavy press work, large castings, foundry	KUKA KR 1000, Fanuc M-2000

The IRC5 Controller: ABB's current generation controller for all IRB robots. It runs the RAPID programming language, manages all six servo drives simultaneously, handles I/O signals, and runs RobotWare — ABB's operating system. Key components: main computer, servo boards, I/O modules, FlexPendant teach pendant, and safety relay unit.

RobotStudio and RAPID Programming

RobotStudio is ABB's offline programming, simulation and commissioning software. It lets you build a complete virtual robot cell, program it in RAPID, simulate the motion, check for collisions, and optimise cycle time — all before touching the real robot. The virtual model is identical to the physical IRC5 controller, so programs transfer without modification.

```

MODULE MainModule

  PERS tooldata tGripper := [TRUE,...];
  PERS wobjdata wStation := [FALSE,...];

  PROC main()
    MoveJ pHome, v500, z50, tGripper!WObj:=wStation;      ← Move instruction
    MoveL pPickup, v200, fine, tGripper!WObj:=wStation;
    SetDO do_GripperClose, 1;  ! Close gripper             ← Digital output
    WaitTime 0.5;
    MoveL pPickup!Offs(0,0,100), v200, z50, tGripper;
    MoveJ pDrop, v500, z50, tGripper!WObj:=wStation;
    SetDO do_GripperClose, 0;  ! Open gripper
    MoveJ pHome, v500, z50, tGripper;
  ENDPROC
ENDMODULE
  
```

Figure 2 — RAPID Program Structure: Module, Procedure, Move and I/O Instructions

RAPID Instruction	Syntax	Function
MoveJ (Joint move)	MoveJ Target, Speed, Zone, Tool [WObj];	Moves all joints simultaneously to target. Fastest motion but unpredictable TCP path. Use for repositioning between welds, picks, or application points.
MoveL (Linear move)	MoveL Target, Speed, Zone, Tool [WObj];	TCP moves in straight line to target. Predictable path — essential for welding, cutting, dispensing, any application requiring exact tool path.
MoveC (Circular move)	MoveC MidPoint, EndPoint, Speed, Zone, Tool;	TCP follows circular arc through MidPoint to EndPoint. Used for circular welds, arcs, and curved dispensing paths.
MoveAbsJ (Absolute joints)	MoveAbsJ [[j1,j2,j3,j4,j5,j6],[eaz]], Speed, Zone, Tool;	Moves each joint to exact angular position. Used for home position and safe service positions.
SetDO (Set digital output)	SetDO Signal_name, Value; (1=ON, 0=OFF)	Sets a digital output signal ON or OFF. Used to control grippers, solenoids, conveyor starts, indicator lights.
WaitDI (Wait for digital input)	WaitDI Signal_name, Value;	Pauses program execution until input signal reaches specified value. Used to wait for sensor confirmation, part-in-place, guard open.
WaitTime (Time delay)	WaitTime Time_seconds;	Pauses for specified number of seconds. Used after gripper close (allow grip to settle), or after spray gun open.

RobotStudio key workflow: 1. Import 3D CAD geometry of robot, fixtures, and parts. 2. Calibrate virtual tool and work object to match physical setup. 3. Teach targets by clicking on the 3D model. 4. Write RAPID program and simulate. 5. Check cycle time and collision. 6. Transfer directly to IRC5 via Ethernet — program runs immediately.

Speed Data, Zone Data, Tool Data and Work Objects

In RAPID, every move instruction requires four mandatory arguments: **target position, speed data, zone data, and tool data**. An optional work object argument specifies the coordinate system for the target. Understanding these arguments is the key to writing professional, production-ready RAPID programs.

SPEED DATA (v)		ZONE DATA (z)	
v5	5 mm/s — precise	fine	Stop exactly at point
v50	50 mm/s — slow	z1	1 mm — very tight blend
v200	200 mm/s — medium	z10	10 mm — slight rounding
v500	500 mm/s — fast	z50	50 mm — smooth corner
v1000	1 000 mm/s — production	z100	100 mm — large blend
vmax	Max robot speed	z200	200 mm — very smooth path

Figure 3 — RAPID Speed Data (v) and Zone Data (z)

RAPID Data Type	Definition	Example Declaration	When to Modify
tooldata (Tool Data)	Defines the tool mounted on the robot flange: TCP position relative to flange, tool mass, and centre of gravity.	PERS tooldata tWeldTorch := [TRUE, [[0,0,180],[1,0,0,0]],[2,[0,0,80],[1,0,0,0],0,0,0]];	Every time you change the end-of-arm tool or replace a worn tool tip
wobjdata (Work Object)	Defines a coordinate frame fixed to the workpiece or fixture. Targets taught relative to this frame move automatically if the fixture moves.	PERS wobjdata wStation1 := [FALSE, TRUE, "", [[500,0,800],[1,0,0,0]],[[0,0,0],[1,0,0,0]]];	When fixture position changes, or for conveyor tracking
speeddata (Speed)	Defines TCP speed (mm/s), reorientation speed (deg/s), and external axes speeds.	PERS speeddata vWeld := [150, 500, 5000, 1000];	When application speed requirements change (weld parameters, new process)
zonedata (Zone / Blend)	Defines how far before the target the robot starts blending toward the next target. "fine" means stop exactly; z50 means blend 50mm early.	CONST zonedata z10 := [FALSE, 10, 15, 15, 1.5, 15, 1.5];	Use "fine" at pick/place points; larger zones for fly-by approach moves

RAPID Best Practice	Why It Matters
Always use named constants for targets, not raw coordinate values	MoveL pPickup is readable and maintainable; MoveL [[450,200,600],...] is not
Use work objects for all workpiece-relative targets	If the fixture moves 50mm, you update one work object — not hundreds of targets
Use "fine" zone only where the robot must stop (grip, release, weld start)	fine adds 200-400ms cycle time per point. Never use fine on approach/departure
Test at low speed first (v50 or v100), increase gradually	Prevents collision damage during initial commissioning of new programs
Add meaningful comments to every non-obvious instruction	Programs are read by maintenance engineers 3 years later who need to understand them

What comes next — ROBOT-005: KUKA Robot Programming — the KUKA KR series robot families, the KRC4 controller, KUKA.Sim simulation software, KRL programming language, and the key differences from ABB RAPID that every multi-brand automation engineer must know.